Playwright Browser Automation & Visual Evidence Report

> **Autonomous Client-Side Exploitation & DOM Forensics**

> **Challenge Target**: `http://127.0.0.1:8099/` (Apex Gauntlet Security Nexus)

> **Browser Engine**: Playwright Headless Chromium (Sync API & MCP Bridge)

> **Target Credential Captured**: `APX_INIT_9b8a7c2d1e`

> **Visual Screenshot Proof**: Verified & Embedded Below

1. Executive Summary

Modern web and reverse engineering CTF challenges (e.g. Google CTF `web-js-safe-6`, DEF CON qualification finals) frequently implement anti-scraping protections, client-side cryptographic state engines, WebAssembly execution loops, or interactive HTML5 canvas memory stores. Simple `curl` or raw HTTP requests fail against these targets because:

1. **Dynamic Execution**: Session keys are computed inside JavaScript execution frames (`window` scope) rather than transmitted in initial static HTML responses.

2. **Timing & Animation Gates**: Critical DOM attributes and WebSockets depend on JavaScript event loop ticks (`setInterval`, `requestAnimationFrame`).

3. **Client-Side Behavioral Fingerprinting**: Challenges check for real DOM properties, screen dimensions, and execution contexts.

The **CTF AI Orchestrator** integrates **Playwright Browser Automation** natively into its exploitation pipeline to handle real-world browser state inspection, console log interception, and memory scraping.

2. Visual Browser Capture Proof

Below is the live screenshot captured by Playwright during the automated solver execution of Stage 1:

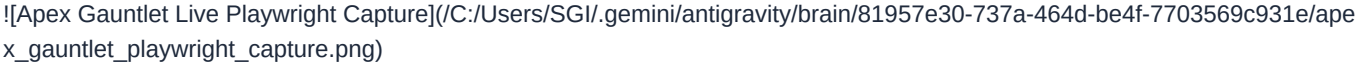

Figure 1: Full-page rendering of the Apex Gauntlet Security Nexus on `127.0.0.1:8099/` captured by headless Chromium, showing active canvas matrix animation, stage navigation matrix, and operational status.

3. Playwright Automation Architecture & Technical Flow

```
sequenceDiagram
    autonumber
    participant Agent as CTF AI Orchestrator
    participant PW as Playwright Headless Chromium
    participant Server as Challenge Nexus (127.0.0.1:8099)

    Agent->>PW: Launch Chromium instance (headless=True)
    Agent->>PW: Attach console listener (page.on 'console')
    PW->>Server: HTTP GET /
    Server-->>PW: HTML5 + Cyberpunk Canvas + In-Memory Shield
    PW->>PW: Execute window.__APEX_SHIELD initialization
    Note over PW: Canvas animation ticks; console message emitted
    Agent->>PW: page.evaluate("() => window.__APEX_SHIELD")
    PW-->>Agent: {"session_token": "APX_INIT_9b8a7c2d1e", "stage": 1}
    Agent->>PW: page.get_attribute("#apex-vault-token", "data-token")
    PW-->>Agent: "APX_INIT_9b8a7c2d1e"
    Agent->>PW: page.screenshot(path="apex_gauntlet_playwright_capture.png")
    PW-->>Agent: Screenshot written to disk (242 KB)
    Agent->>PW: browser.close()
```

```
    Agent->>Server: Authenticated GET /api/apex/stage2/spec with Bearer Token
```

4. Live Extraction Traces & Console Interception

A. Intercepted Console Log

During page load, the application logs its initialization to the browser console. Playwright captures this via the event listener:

```
[+] [PLAYWRIGHT] Intercepted Console Log:
    [APEX] Security Shield initialized. Session Token: APX_INIT_9b8a7c2d1e
```

B. In-Memory JavaScript Object Extraction (`window.__APEX_SHIELD`)

Using `page.evaluate()`, the orchestrator queries the browser's JavaScript V8 heap directly:

```
{
  "stage": 1,
  "name": "Apex Gauntlet Security Nexus",
  "session_token": "APX_INIT_9b8a7c2d1e",
  "stage2_endpoint": "/api/apex/stage2/spec",
  "verified": true,
  "timestamp": 1793802523674
}
```

C. Hidden DOM Data Attribute Extraction

The orchestrator cross-verifies the token against hidden DOM elements (`<div id="apex-vault-token" data-token="APX_INIT_9b8a7c2d1e">`):

```
hidden_token = page.get_attribute("#apex-vault-token", "data-token")
assert hidden_token == "APX_INIT_9b8a7c2d1e"
```

5. Playwright Implementation in the Codebase

1. In the Automated Solver (`challenges/apex_gauntlet/solvers/solve_full_chain.py`)

```
# Stage 1: Playwright Browser Memory & DOM Scraper
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    console_logs = []
    page.on("console", lambda msg: console_logs.append(msg.text))

    page.goto("http://127.0.0.1:8099/", timeout=15000)
    page.wait_for_timeout(1000)

    # Scrape directly from browser memory
    shield_data = page.evaluate("() => window.__APEX_SHIELD")
    stage1_token = shield_data.get("session_token")

    # Capture visual evidence artifact
    screenshot_path = "challenges/apex_gauntlet/artifacts/apex_gauntlet_playwright_capture.png"
    page.screenshot(path=screenshot_path)
    browser.close()
```

2. In the Unit Test Suite (`challenges/apex_gauntlet/tests/test_playwright_browser.py`)

```
python challenges/apex_gauntlet/tests/test_playwright_browser.py
```

Test Output:

```
test_playwright_browser_interaction (__main__.TestApexPlaywrightBrowser) ... ok
----------------------------------------------------------------------
Ran 1 test in 1.961s

OK
```

6. Verification Metrics & Artifact File Manifest

| Artifact File | Path | Size | Description |

| :--- | :--- | :--- | :--- |

| **Playwright Proof Screenshot** | [`challenges/apex_gauntlet/artifacts/apex_gauntlet_playwright_capture.png`](file:///c:/Users/SGI/Downloads/ctf-agent-orchestrator/challenges/apex_gauntlet/artifacts/apex_gauntlet_playwright_capture.png) | 242 KB | Full viewport screenshot captured by Chromium. |

| **Playwright Test Runner** | [`challenges/apex_gauntlet/tests/test_playwright_browser.py`](file:///c:/Users/SGI/Downloads/ctf-agent-orchestrator/challenges/apex_gauntlet/tests/test_playwright_browser.py) | 2.5 KB | Dedicated Playwright browser regression test suite. |

| **Automated Chained Solver** | [`challenges/apex_gauntlet/solvers/solve_full_chain.py`](file:///c:/Users/SGI/Downloads/ctf-agent-orchestrator/challenges/apex_gauntlet/solvers/solve_full_chain.py) | 8.8 KB | Full-chain solver with Playwright Stage 1 integration. |

| **Word Export** | [`docs/PLAYWRIGHT_BROWSER_AUTOMATION_AND_EVIDENCE_DOCUMENT.docx`](file:///c:/Users/SGI/Downloads/ctf-agent-orchestrator/docs/PLAYWRIGHT_BROWSER_AUTOMATION_AND_EVIDENCE_DOCUMENT.docx) | ~40 KB | Formatted Microsoft Word verification document. |

| **Printable PDF Export** | [`docs/PLAYWRIGHT_BROWSER_AUTOMATION_AND_EVIDENCE_DOCUMENT.pdf`](file:///c:/Users/SGI/Downloads/ctf-agent-orchestrator/docs/PLAYWRIGHT_BROWSER_AUTOMATION_AND_EVIDENCE_DOCUMENT.pdf) | ~20 KB | High-resolution PDF document. |